ESE5023 Assignment 02

1、Significant earthquakes since 2150 B.C.

思路：

文件是 tsv 格式，第一行是列名，第二行是下载参数占位行 "[]"，真正的数据从第三行开始。

用 pandas 读取 tsv；指定制表符分隔；跳过第二行，确保数字列被识别成数字。

查看行数和列数，前五行，检查是否读取正确。

这个文件共有 6697 条地震记录、162 个国家。

1.1 各国累计死亡人数

题目理解：按国家把 Total Deaths 相加，再按总死亡人数从大到小排列，输出前 20 个国家。

分析思路：

groupby("Country") 按国家分组；

Sum() 把每组的死亡人数相加；

sort_values() 降序排序；

head(20) 取前 20 行。

```
[5]: deaths_by_country = (
         Sig_Eqs.groupby("Country", as_index=False)["Total Deaths"]
         .sum()
     )

     deaths_by_country = deaths_by_country.sort_values(
         "Total Deaths",
         ascending=False
     )

     top20_deaths = deaths_by_country.head(20).copy()

     top20_deaths["Total Deaths"] = (
         top20_deaths["Total Deaths"]
         .astype(int)
     )

     top20_deaths = top20_deaths.reset_index(drop=True)
     top20_deaths.index = range(1, len(top20_deaths) + 1)

     print(top20_deaths)
```

	Country	Total Deaths
1	CHINA	2106526
2	TURKEY	1008867
3	IRAN	761656
4	SYRIA	488326
5	ITALY	417297
6	HAITI	323782
7	JAPAN	319510
8	AZERBAIJAN	319251
9	INDONESIA	283485
10	ARMENIA	175000
11	PAKISTAN	143714
12	ECUADOR	134445
13	TURKMENISTAN	110412
14	PERU	97009
15	GREECE	83658
16	PORTUGAL	82572
17	IRAQ	70200
18	CHILE	70175
19	INDIA	63413
20	ISRAEL	58368

1.2 每年 Ms > 3.0 的地震数量

题目理解：只看 Ms 列，保留严格大于 3.0 的记录，再按年份统计每年有多少条，最后画时间序列。

分析思路：先转换 Ms，过滤空值和无效值；再用 Ms > 3.0 筛选；然后按 Year 计数；最后补齐没有合格事件的年份。

errors="coerce"会把空值或无法转换的内容变成 NaN，避免程序报错。

按 Year 年份分组。

.size() 统计每一年有多少条满足 Ms > 3.0 的记录。

结果是每年对应的地震数量。
annual_counts.plot()：年份作为横轴，地震数量作为纵轴，绘制时间序列。
plt.title(...)：设置图标题。
plt.xlabel(...)：设置横轴名称。
plt.ylabel(...)：设置纵轴名称。

```
[7]: import pandas as pd
     import matplotlib.pyplot as plt

     ms = pd.to_numeric(
         Sig_Eqs["Ms"],
         errors="coerce"
     )

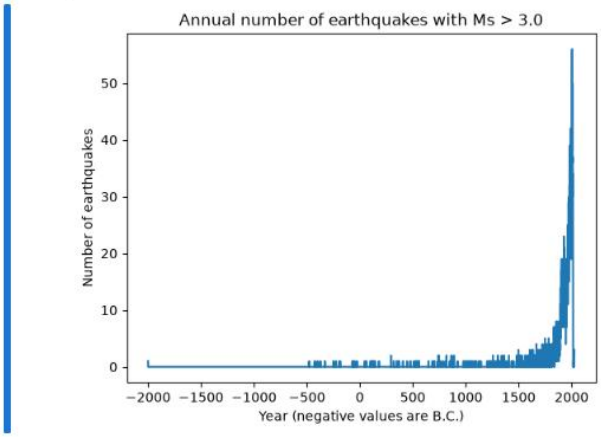
     ms_events = Sig_Eqs.loc[ms > 3.0]

     annual_counts = ms_events.groupby("Year").size()

     year_range = range(
         int(ms_events["Year"].min()),
         int(ms_events["Year"].max()) + 1
     )

     annual_counts = annual_counts.reindex(
         year_range,
         fill_value=0
     )

     annual_counts.plot()
     plt.title("Annual number of earthquakes with Ms > 3.0")
     plt.xlabel("Year (negative values are B.C.)")
     plt.ylabel("Number of earthquakes")
     plt.show()
```



Do you observe any trend? Explain why or why not?

从时间序列图可以观察到，Ms > 3.0 的记录数量在早期较低，进入 20 世纪后明显增加，最大峰值出现在横轴最右侧约 2000 年附近，随后快速下降。由于横轴时间跨度过大，图中无法直接辨认具体峰值年份。但是，这不能解释为全球真实地震频率先增加后减少。该数据库只收录显著或破坏性地震，并非全球所有地震的完整目录。随着历史发展，观测、通信、人口暴露和灾害报告能力不断提高，因此早期地震更容易被漏记。同时，还有可能记录缺少 Ms 数据，这也可能会导致统计数量明显下降。

因此，图中的趋势主要反映数据覆盖范围、报告情况和震级数据可用性的变化，不能作为全球真实地震活动变化的证据。

1.3 地震总数与最大地震

题目理解：写一个函数，输入国家名称，返回该国地震总数、最大地震的日期和地点。最后应用于全部国家，并按地震数量降序输出。

分析思路：先筛选某一个国家的数据；用行数得到地震总数；按 Mag 找最大地震（综合了 Mw、Ms、Mb、Ml 等不同震级类型，为每条地震记录提供一个可比较的震级值）；格

式化日期和地点；最后循环所有国家。

sort_values("Mag", ascending=False)：按震级从大到小排序

abs(year) 把负数转换成正数

按 Total Earthquakes 列排序

[8]:
```
def CountEq_LargestEq(country):
    country_events = Sig_Eqs.loc[
        Sig_Eqs["Country"] == country
    ]

    total_count = len(country_events)

    if total_count == 0:
        return 0, None, None

    events_with_mag = country_events.dropna(
        subset=["Mag"]
    )

    if events_with_mag.empty:
        return total_count, None, None

    largest = events_with_mag.sort_values(
        "Mag",
        ascending=False
    ).iloc[0]

    year = int(largest["Year"])

    if year < 0:
        event_date = f"{abs(year)} B.C."
    else:
        event_date = str(year)

    if not pd.isna(largest["Mo"]):
        month = int(largest["Mo"])
        event_date = f"{event_date}-{month:02d}"

        if not pd.isna(largest["Dy"]):
            day = int(largest["Dy"])
            event_date = f"{event_date}-{day:02d}"

    location = largest["Location Name"]

    return total_count, event_date, location


results = []

for country in Sig_Eqs["Country"].dropna().unique():
    count, date, location = CountEq_LargestEq(country)

    results.append({
        "Country": country,
        "Total Earthquakes": count,
        "Largest Earthquake Date": date,
        "Largest Earthquake Location": location
    })

country_summary = pd.DataFrame(results)

country_summary = country_summary.sort_values(
    "Total Earthquakes",
    ascending=False
).reset_index(drop=True)

country_summary.index = range(
    1,
    len(country_summary) + 1
)

print(country_summary)
```

```
           Country  Total Earthquakes Largest Earthquake Date  \
1            CHINA                626             1668-07-25
2            JAPAN                429             2011-03-11
3        INDONESIA                427             2004-12-26
4             IRAN                385              856-12-22
5           TURKEY                359             1939-12-26
..             ...                ...                    ...
158        BURUNDI                  1             2004-02-24
159  CZECH REPUBLIC                 1             2008-11-22
160     MADAGASCAR                  1             2017-01-11
161         ZAMBIA                  1             2017-02-24
162        COMOROS                  1             2018-05-15

                    Largest Earthquake Location
1                     CHINA:  SHANDONG PROVINCE
2                              JAPAN:  HONSHU
3    INDONESIA:  SUMATRA:  ACEH:  OFF WEST COAST
4                          IRAN:  DAMGHAN, QUMIS
5                              TURKEY:  ERZINCAN
..                                           ...
158                         BURUNDI:  RUYAGA
159                 CZECH REPUBLIC:  KARVINA
160                    MADAGASCAR: ANTSIRABE
161                         ZAMBIA:  KAPUTA
162                        COMOROS:  MAYOTTE

[162 rows x 4 columns]
```

2、Air temperature in Shenzhen 1998-2022

Explain how you filter the data in your report.

根据《综合用户指南》第 10-11 页，TMP 位于 POS 88-92，表示空气温度，单位为摄氏度，缩放因子为 10，因此原始值+0186 应转换为 18.6℃，同时该字段的合理范围为-93.2℃至 61.8℃，而+9999 表示温度缺失；POS 93-93 是温度质量控制码，其中 2、3、6、7 分别表示可疑或错误数据，0、1、4、5、9 表示通过相应质量检查，A、C、I、M、P、R、U 表示数据被接受、人工修改、插入或替换。因此，我的筛选方法是：（1）将 TMP 按逗号拆分为温度原始值和质量控制码两部分；（2）将温度原始值除以 10，转换为摄氏度；（3）删除温度缺失记录，包括+9999；（4）删除低于-93.2℃或高于 61.8℃的异常温度；（5）删除质量控制码为 2、3、6、7 的可疑或错误数据；（6）保留质量控制码为 0、1、4、5、9 以及 A、C、I、M、P、R、U 的记录。

二、分析思路

只读取 DATE 和 TMP 两列；把 DATE 转成 pandas 日期格式；把 TMP 按逗号拆成两列：温度原始值和质量控制码；温度原始值除以 10，得到摄氏度；过滤：无效日期，缺失温度，超出用户指南合理范围-93.2℃到 61.8℃的值，质量控制码为 2、3、6、7 的可疑或错误数据；按年月分组，计算每月平均温度；用一元线性回归拟合长期趋势；画月度温度曲线和线性趋势线。

```
[3]: from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

CSV_FILE = Path(
    r"C:\Users\admin\Baoan_Weather_1998_2022.csv"
)

PDF_FILE = Path(
    r"C:\Users\admin\isd-format-document.pdf"
)

print("CSV exists:", CSV_FILE.exists())
print("PDF exists:", PDF_FILE.exists())

weather = pd.read_csv(
    CSV_FILE,
    usecols=["DATE", "TMP"],
)

weather["DATE"] = pd.to_datetime(
    weather["DATE"],
    errors="coerce"
)

tmp_parts = weather["TMP"].str.split(
    ",",
    expand=True
)

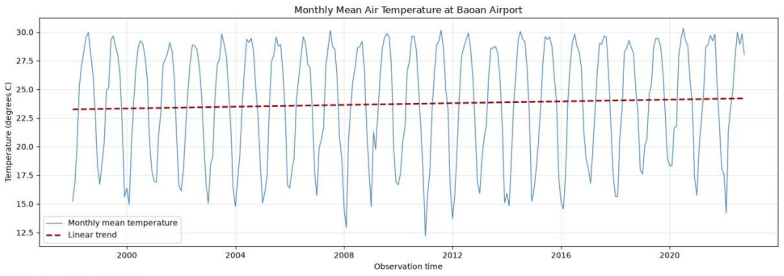
weather["TEMP C"] = (
plt.plot(
    monthly_temperature.index,
    monthly_temperature.values,
    color="steelblue",
    linewidth=1,
    label="Monthly mean temperature"
)

plt.plot(
    monthly_temperature.index,
    trend_line,
    color="darkred",
    linewidth=2,
    linestyle="--",
    label="Linear trend"
)

plt.title("Monthly Mean Air Temperature at Baoan Airport")
plt.xlabel("Observation time")
plt.ylabel("Temperature (degrees C)")
plt.grid(alpha=0.3)
plt.legend()
plt.tight_layout()
plt.show()

print("Valid observations:", len(weather))
print("Monthly points:", len(monthly_temperature))
print("Temperature range:", weather["TEMP_C"].min(), weather["TEMP_C"].max())
print("Linear trend:", slope_per_year, "degrees C per year")
print("Trend per decade:", slope_per_year * 10, "degrees C")
print("Correlation:", correlation)
```

```
CSV exists: True
PDF exists: True
```




```
Valid observations: 234864
Monthly points: 298
Temperature range: 1.8 38.0
Linear trend: 0.03881566936867783 degrees C per year
Trend per decade: 0.3881566936867783 degrees C
Correlation: 0.05568031128709581
```

Is there a trend in monthly averaged air temperature from 1998 to 2022?

1998 至 2022 年，宝安机场月平均空气温度总体呈弱上升趋势。对全部月平均温度进行线性拟合，趋势约为+0.039℃/年，即约+0.39°C/十年。由于数据存在明显的季节循环和年际波动，相关系数和 R^2较低，说明季节变化远大于长期趋势。

3、Global collection of hurricanes

一、题目理解

IBTrACS 是国际最佳路径气候管理档案，每一行表示某个热带气旋在某一时刻的位置和强度记录。

题目中的 SID 是风暴编号，同一个 SID 的所有行属于同一个风暴；WMO_WIND 是风速，单位为节；LAT、LON 分别是纬度和经度；BASIN 是海盆。

读取 CSV 时要使用以下设置：

usecols=range(17)：只读取需要的 17 列，减少内存占用。

skiprows=[1, 2]：跳过单位行和变量说明行。

parse_dates=["ISO_TIME"]：把时间转换成 pandas 日期时间。
na_values=["NOT_NAMED", " "]：把未命名和空白识别为缺失值。
low_memory=False：避免分块读取时出现类型警告。

二、分析思路

3.1 和 3.2：先按 SID 找出每个风暴的最大 WMO_WIND，再对风暴峰值排序。
3.3：对 BASIN 做频数统计，并保留缺失海盆。
3.4：用经纬度绘制 hexbin 图，颜色表示六边形内的数据点数量。
3.5：同时筛选 2018 年和 MANGKHUT，按时间排序后绘制路径。
3.6：筛选 1970 年以来的 WP 和 EP 海盆。
3.7：用 resample("D").size() 计算每日数据点数量。
3.8：从日期提取 day of year，按日序求多年平均。
3.9：每日数据点数量减去对应日序的气候态。
3.10：按年平均异常，并排除不足 360 天的年份。

[4]:
```python
import pandas as pd
import matplotlib.pyplot as plt

#1. 设置数据文件路径

DATA_FILE = r"C:\Users\admin\ibtracs.ALL.list.v04r00.csv"

#2. 读取数据

df = pd.read_csv(
    DATA_FILE,
    usecols=range(17),
    skiprows=[1, 2],
    parse_dates=["ISO_TIME"],
    na_values=["NOT_NAMED", " "],
    low_memory=False,
)

df["NAME"] = df["NAME"].fillna("UNNAMED")
df["BASIN"] = df["BASIN"].fillna("UNKNOWN")
df["LAT"] = pd.to_numeric(df["LAT"], errors="coerce")
df["LON"] = pd.to_numeric(df["LON"], errors="coerce")
df["WMO_WIND"] = pd.to_numeric(df["WMO_WIND"], errors="coerce")

# 没有时间或位置的记录不能用于轨迹分析
df = df.dropna(subset=["ISO_TIME", "LAT", "LON"]).copy()

print("数据形状：", df.shape)
print("开始时间：", df["ISO_TIME"].min())
print("结束时间：", df["ISO_TIME"].max())
```

```
数据形状： (707176, 17)
开始时间： 1842-10-25 06:00:00
结束时间： 2022-10-13 00:00:00
```

[ ]:
```python
#3.1和3.2找出风速最大的风暴
```

[5]:
```python
#没有风速的记录不能参与比较
wind_data = df.dropna(subset=["WMO_WIND"]).copy()

#先按风速从大到小排序
wind_data = wind_data.sort_values(
    ["WMO_WIND", "ISO_TIME"],
    ascending=[False, True],
)

#同一个SID只保留第一行，也就是峰值那一行
storm_peak = wind_data.drop_duplicates(
    subset="SID",
    keep="first",
)

print("\n3.1 风速最大的 10 个风暴：")
print(
    storm_peak[
        ["SEASON", "NAME", "SID", "WMO_WIND"]
    ]
    .head(10)
    .to_string(index=False)
)

print("\n3.2 最强 20 个风暴的峰值风速：")
print(
    storm_peak["WMO_WIND"]
    .head(20)
    .to_string(index=False)
)
```

```
3.1 风速最大的 10 个风暴：
 SEASON     NAME           SID  WMO_WIND
   2015 PATRICIA 2015293N13266     185.0
   1980    ALLEN 1980214N11330     165.0
   1935  UNNAMED 1935241N23291     160.0
   1988  GILBERT 1988253N12306     160.0
   1997    LINDA 1997253N12255     160.0
   2005    WILMA 2005289N18282     160.0
   2019   DORIAN 2019236N10314     160.0
   1998    MITCH 1998295N12284     155.0
   2005     RITA 2005261N21290     155.0
   2009     RICK 2009288N07267     155.0

3.2 最强 20 个风暴的峰值风速：
185.0
165.0
160.0
160.0
160.0
160.0
160.0
155.0
155.0
155.0
155.0
150.0
150.0
150.0
150.0
150.0
150.0
150.0
150.0
150.0
```

[10]:
```
#3.3 统计每个海盆有多少条数据
basin_counts = df["BASIN"].value_counts(dropna=False)

print("\n3.3 各海盆的数据点数量：")
print(basin_counts)
```

```
3.3 各海盆的数据点数量：
BASIN
WP         236576
SI         160668
UNKNOWN    124881
SP          67119
EP          62412
NI          55401
SA            119
Name: count, dtype: int64
```

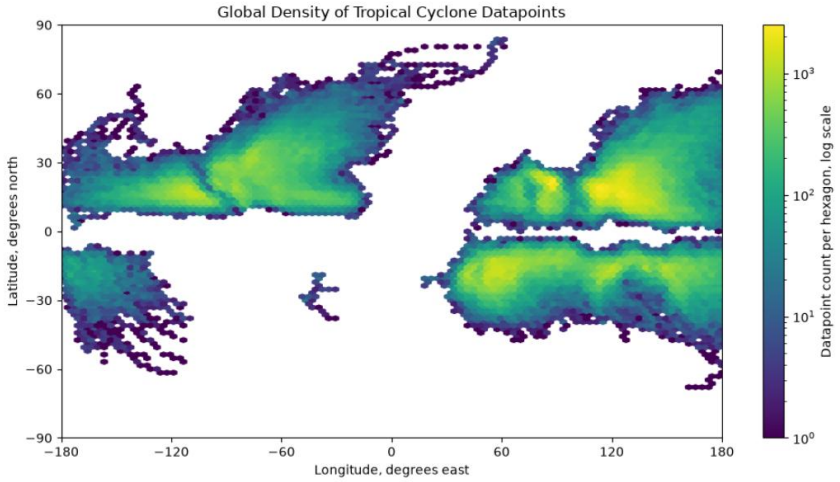
[11]:
```
#3.4 绘制经纬度 hexbin 密度图

plt.figure(figsize=(10, 5.5))

hexbin = plt.hexbin(
    df["LON"],
    df["LAT"],
    gridsize=100,
    extent=(-180, 180, -90, 90),
    mincnt=1,
    bins="log",
    cmap="viridis",
)

plt.colorbar(
    hexbin,
    label="Datapoint count per hexagon, log scale",
)

plt.title("Global Density of Tropical Cyclone Datapoints")
plt.xlabel("Longitude, degrees east")
plt.ylabel("Latitude, degrees north")
plt.xlim(-180, 180)
plt.ylim(-90, 90)
plt.xticks(range(-180, 181, 60))
plt.yticks(range(-90, 91, 30))
plt.tight_layout()
plt.show()
```



[12]:
```python
#3.5 绘制 2018 年 Mangkhut 的路径

mangkhut = df[
    (df["SEASON"] == 2018)
    & (df["NAME"].str.upper() == "MANGKHUT")
].copy()

# 不排序的话，轨迹线可能乱连
mangkhut = mangkhut.sort_values("ISO_TIME")

if mangkhut.empty:
    raise ValueError("没有找到 2018 年的 Mangkhut")

start = mangkhut.iloc[0]
end = mangkhut.iloc[-1]

print("\n3.5 Mangkhut 数据点数量：", len(mangkhut))
print("起点：", start["ISO_TIME"], start["LAT"], start["LON"])
print("终点：", end["ISO_TIME"], end["LAT"], end["LON"])
print("最大风速：", mangkhut["WMO_WIND"].max())

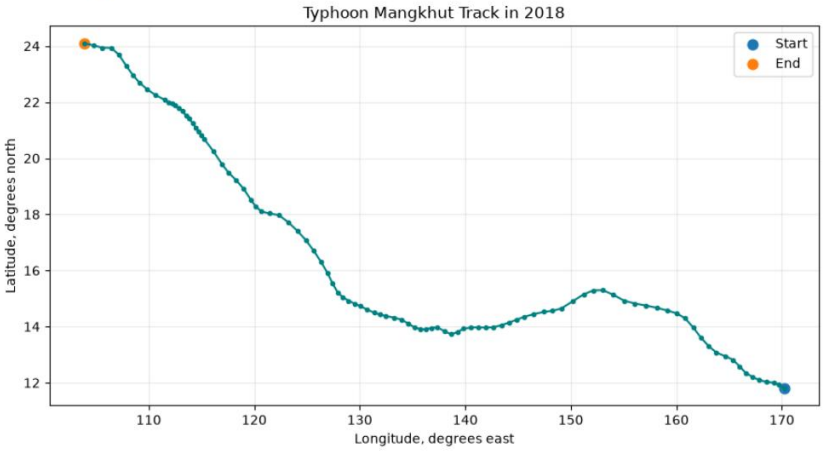
plt.figure(figsize=(9, 5))

plt.plot(
    mangkhut["LON"],
    mangkhut["LAT"],
    color="teal",
    marker="o",
    markersize=3,
    linewidth=1.5,
)

plt.scatter(start["LON"], start["LAT"], s=60, label="Start")
plt.scatter(end["LON"], end["LAT"], s=60, label="End")

plt.title("Typhoon Mangkhut Track in 2018")
plt.xlabel("Longitude, degrees east")
plt.ylabel("Latitude, degrees north")
plt.grid(alpha=0.25)
plt.legend()
plt.tight_layout()
plt.show()
```

```
3.5 Mangkhut 数据点数量： 99
起点： 2018-09-06 12:00:00 11.8 170.2
终点： 2018-09-17 18:00:00 24.1 103.9
最大风速： 110.0
```



[13]:
```python
#3.6 筛选 1970 年以来的 WP 和 EP

filtered = df[
    (df["SEASON"] >= 1970)
    & df["BASIN"].isin(["WP", "EP"])
].copy()

print("\n3.6 筛选后的数据形状：", filtered.shape)
print(filtered["BASIN"].value_counts())
```

```
3.6 筛选后的数据形状： (176352, 17)
BASIN
WP    122301
EP     54051
Name: count, dtype: int64
```

[14]:
```
#3.7 统计每天有多少个数据点

# 把时间设为索引，才能使用 resample
filtered = filtered.set_index("ISO_TIME")

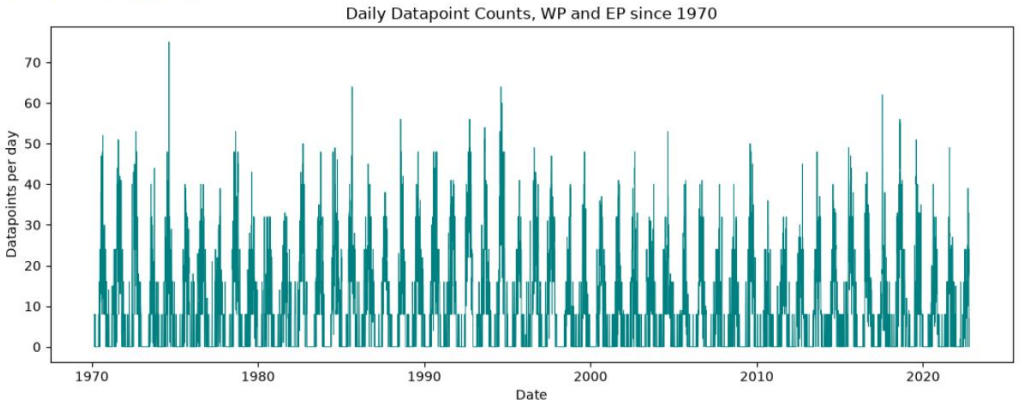
# D 表示按日分组，size() 表示统计每组有多少行
daily_counts = filtered.resample("D").size()
daily_counts.name = "daily_count"

print("\n3.7 每日数据点数量统计：")
print(daily_counts.describe())


# 建立一张简单的日数据表
daily = pd.DataFrame({
    "date": daily_counts.index,
    "count": daily_counts.values,
})

plt.figure(figsize=(11, 4.5))
plt.plot(daily["date"], daily["count"], linewidth=0.6, color="teal")
plt.title("Daily Datapoint Counts, WP and EP since 1970")
plt.xlabel("Date")
plt.ylabel("Datapoints per day")
plt.tight_layout()
plt.show()
```

```
3.7 每日数据点数量统计：
count    19229.000000
mean         9.171148
std         11.148994
min          0.000000
25%          0.000000
50%          8.000000
75%         16.000000
max         75.000000
Name: daily_count, dtype: float64
```



[15]:
```
#3.8 按 day of year 计算气候态

# dayofyear：1 月 1 日是 1，12 月 31 日是 365 或 366
daily["day_of_year"] = daily["date"].dt.dayofyear

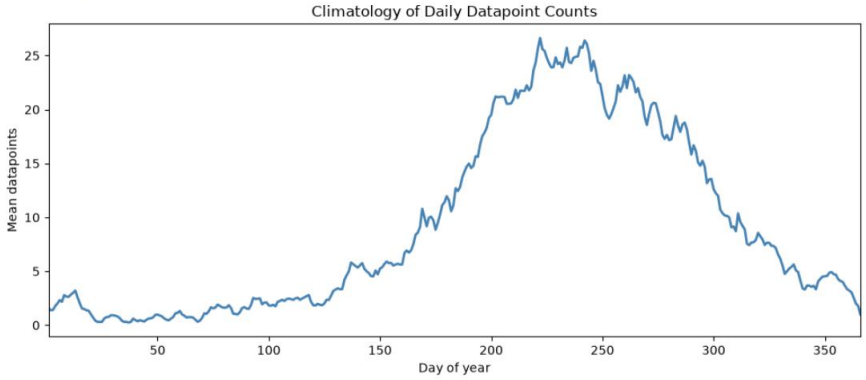
# 把所有年份中同一个日序的计数求平均
climatology = daily.groupby("day_of_year")["count"].mean()

print("\n3.8 气候态最大值：")
print("day_of_year =", climatology.idxmax())
print("平均值 =", climatology.max())

plt.figure(figsize=(10, 4.5))
plt.plot(
    climatology.index,
    climatology.values,
    color="steelblue",
    linewidth=2,
)

plt.title("Climatology of Daily Datapoint Counts")
plt.xlabel("Day of year")
plt.ylabel("Mean datapoints")
plt.xlim(1, 366)
plt.tight_layout()
plt.show()
```

```
3.8 气候态最大值：
day_of_year = 222
平均值 = 26.62264150943396
```



[16]:
```
#3.9 计算逐日异常

# 找出每一天对应的气候态
daily["climatology"] = daily["day_of_year"].map(climatology)

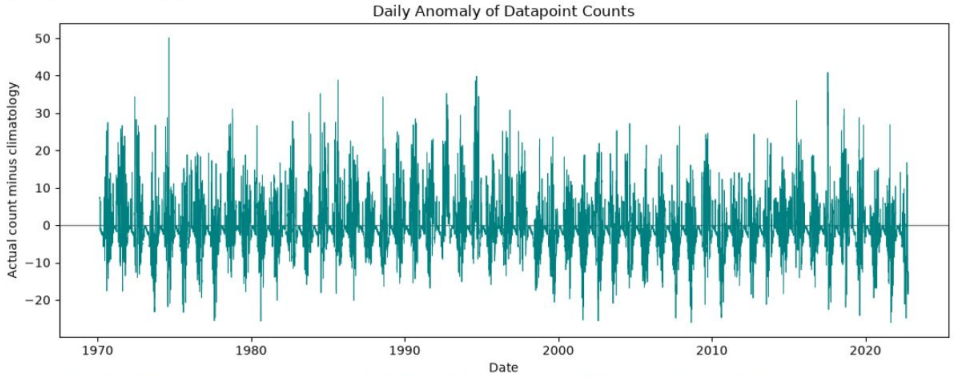
# 异常 = 当天实际值 - 同一天序的气候态
daily["anomaly"] = daily["count"] - daily["climatology"]

print("\n3.9 逐日异常统计：")
print(daily["anomaly"].describe())

plt.figure(figsize=(11, 4.5))
plt.axhline(0, color="gray", linewidth=1)
plt.plot(
    daily["date"],
    daily["anomaly"],
    color="teal",
    linewidth=0.6,
)

plt.title("Daily Anomaly of Datapoint Counts")
plt.xlabel("Date")
plt.ylabel("Actual count minus climatology")
plt.tight_layout()
plt.show()
```

```
3.9 逐日异常统计：
count    19229.000000
mean         0.000000
std          7.389795
min        -26.094340
25%         -3.584906
50%         -1.019231
75%          3.500000
max         50.113208
Name: anomaly, dtype: float64
```



[17]:
```python
# 3.10 计算年度异常并画图

# 把日期设回索引，再按年求平均异常
daily_by_date = daily.set_index("date")

annual_anomaly = daily_by_date["anomaly"].resample("YS").mean()

# 统计每年有多少天
days_per_year = daily_by_date.groupby(daily_by_date.index.year).size()

# 只有至少 360 天的年份才参与排名
complete_years = days_per_year[days_per_year >= 360].index

annual_complete = annual_anomaly[
    annual_anomaly.index.year.isin(complete_years)
]

# 把年度 Series 转成表格，方便查看
annual_table = annual_complete.reset_index()
annual_table.columns = ["date", "annual_anomaly"]
annual_table["year"] = annual_table["date"].dt.year

print("\n3.10 年度异常最高的 5 年：")
print(
    annual_table
    .sort_values("annual_anomaly", ascending=False)
    .head(5)
    .to_string(index=False)
)

print("\n3.10 年度异常最低的 5 年：")
print(
    annual_table
    .sort_values("annual_anomaly")
    .head(5)
    .to_string(index=False)
)

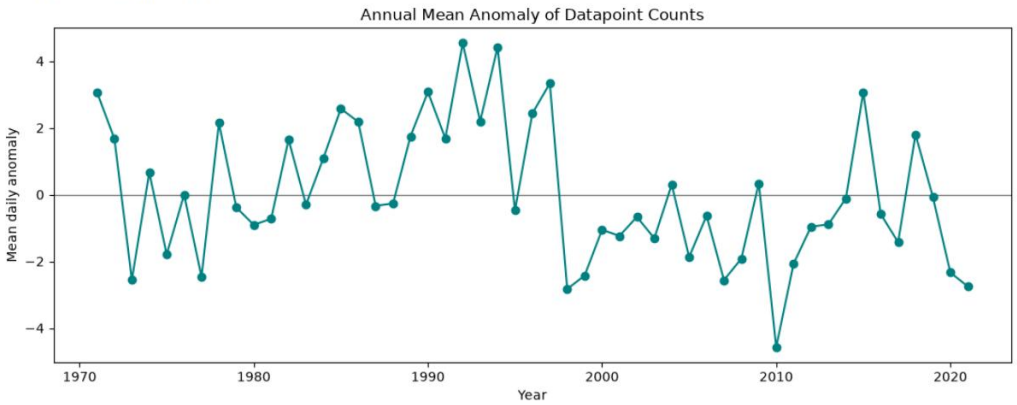
plt.figure(figsize=(11, 4.5))
plt.axhline(0, color="gray", linewidth=1)

plt.plot(
    annual_complete.index.year,
    annual_complete.values,
    color="teal",
    marker="o",
    linewidth=1.5,
)

plt.title("Annual Mean Anomaly of Datapoint Counts")
plt.xlabel("Year")
plt.ylabel("Mean daily anomaly")
plt.tight_layout()
plt.show()
```

```
3.10 年度异常最高的 5 年：
      date  annual_anomaly  year
1992-01-01        4.537501  1992
1994-01-01        4.415686  1994
1997-01-01        3.330754  1997
1990-01-01        3.073220  1990
2015-01-01        3.067741  2015

3.10 年度异常最低的 5 年：
      date  annual_anomaly  year
2010-01-01       -4.570616  2010
1998-01-01       -2.830890  1998
2021-01-01       -2.751437  2021
2007-01-01       -2.562396  2007
1973-01-01       -2.556917  1973
```



年度异常明显偏高的年份是 1992、1994、1997；年度异常明显偏低的年份是 2010、1998、2021。其中 1992 年最高，2010 年最低。

4、Explore a data set

下载了

https://www.ncei.noaa.gov/access/search/datasets/global-summary-of-the-month/的 VINING 0.2 ENE, IA US。这份数据适合分析月总降水量 PRCP

一、题目理解

4.1 要求加载数据并清洗缺失值和异常值。

这份数据每个 DATE 代表一个月，适合分析月总降水量 PRCP。同时可以辅助使用：

DP01：降水量大于 0.01 英寸的日数。

DP10：降水量大于 0.10 英寸的日数。

DP1X：降水量大于 1.00 英寸的日数。

EMXP：当月最大单日降水量。

4.2 要求画一个变量的时间序列。使用 DATE 作为横轴，PRCP 作为纵轴。

4.3 至少做五项统计检查。代码会完成：

1. 数据完整性和缺失月份检查。
2. 描述统计和变异程度检查。
3. 月份季节性检查。
4. 长期线性趋势检查。
5. 月降水量与强降水日数的相关性检查。

二、分析思路

先读取与降水相关的少量列，不读取文件中 100 多个基本为空的字段。然后转换日期和数值类型，检查缺失、重复、负降水量、极端日降水量大于月总量、降水日数超过当月天数等质量问题。清洗后绘制月降水量时间序列，再依次完成统计表、季节柱状图、趋势拟合和相关性计算。

[1]:
```
from pathlib import Path
import calendar

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from IPython.display import display

# 数据文件路径
DATA_FILE = Path(r"C:\Users\admin\US1IATM0004.csv")
# 只读取需要的列
columns = ["DATE", "PRCP", "DP01", "DP10", "DP1X", "EMXP"]

df = pd.read_csv(DATA_FILE, usecols=columns)

# 显示数据规模和前五行
print("原始数据形状：", df.shape)
display(df.head())
```

原始数据形状： (87, 6)

	DATE	DP01	DP10	DP1X	EMXP	PRCP
0	2019-07	6	6	0	22.6	84.6
1	2019-08	8	6	2	46.0	121.1
2	2019-09	14	14	2	31.0	179.7
3	2019-10	11	11	2	54.6	182.2
4	2019-11	7	7	0	9.1	43.6

[ ]:
```
#4.1 数据清洗
```

[2]:
```
# DATE 形如 2019-07，转换成每月第一天
df["DATE"] = pd.to_datetime(
    df["DATE"],
    format="%Y-%m",
    errors="coerce",
)

# 统一把数值列转换成数值类型
for col in ["PRCP", "DP01", "DP10", "DP1X", "EMXP"]:
    df[col] = pd.to_numeric(df[col], errors="coerce")

# 记录清洗前信息
raw_rows = len(df)
missing_date = df["DATE"].isna().sum()
missing_prcp = df["PRCP"].isna().sum()
duplicate_months = df["DATE"].duplicated().sum()

# 删除日期或 PRCP 缺失的记录
df = df.dropna(subset=["DATE", "PRCP"])

# 删除重复月份
df = df.drop_duplicates(subset=["DATE"], keep="last")

# 找出不合理记录
invalid = (
    (df["PRCP"] < 0)
    | (df["EMXP"] < 0)
    | (df["EMXP"] > df["PRCP"])
    | (df["DP10"] > df["DP01"])
    | (df["DP1X"] > df["DP10"])
    | (df["DP01"] > df["DATE"].dt.days_in_month)
)

# 保存异常值数量
invalid_count = int(invalid.sum())

# 删除异常记录并按时间排序
df = df.loc[~invalid].sort_values("DATE").reset_index(drop=True)

# 检查月份是否连续
expected_months = pd.date_range(
    start=df["DATE"].min(),
    end=df["DATE"].max(),
    freq="MS",
)
missing_months = expected_months.difference(df["DATE"])
completeness = len(df) / len(expected_months)

# 用一张表展示清洗结果
cleaning_report = pd.DataFrame(
    {
        "项目": [
            "原始行数",
            "DATE 缺失",
            "PRCP 缺失",
            "重复月份",
            "异常记录",
            "清洗后行数",
            "起始月份",
            "结束月份",
            "缺失月份数",
            "数据完整率",
        ],
        "结果": [
            raw_rows,
            missing_date,
            missing_prcp,
            duplicate_months,
            invalid_count,
            len(df),
            df["DATE"].min().strftime("%Y-%m"),
            df["DATE"].max().strftime("%Y-%m"),
            len(missing_months),
            f"{completeness:.2%}",
        ],
    }
)

display(cleaning_report)
display(df.head())
```

	项目	结果
0	原始行数	87
1	DATE 缺失	0
2	PRCP 缺失	0
3	重复月份	0
4	异常记录	0
5	清洗后行数	87
6	起始月份	2019-07
7	结束月份	2026-09
8	缺失月份数	0
9	数据完整率	100.00%

	DATE	DP01	DP10	DP1X	EMXP	PRCP
0	2019-07-01	6	6	0	22.6	84.6
1	2019-08-01	8	6	2	46.0	121.1
2	2019-09-01	14	14	2	31.0	179.7
3	2019-10-01	11	11	2	54.6	182.2
4	2019-11-01	7	7	0	9.1	43.6

[ ]:
```
#4.2 月降水量时间序列
```

[3]:
```
# 设置日期索引
monthly = df.set_index("DATE")["PRCP"]

# 12 个月移动平均
rolling_12 = monthly.rolling(12).mean()

# 创建图形
fig, ax = plt.subplots(figsize=(12, 5))

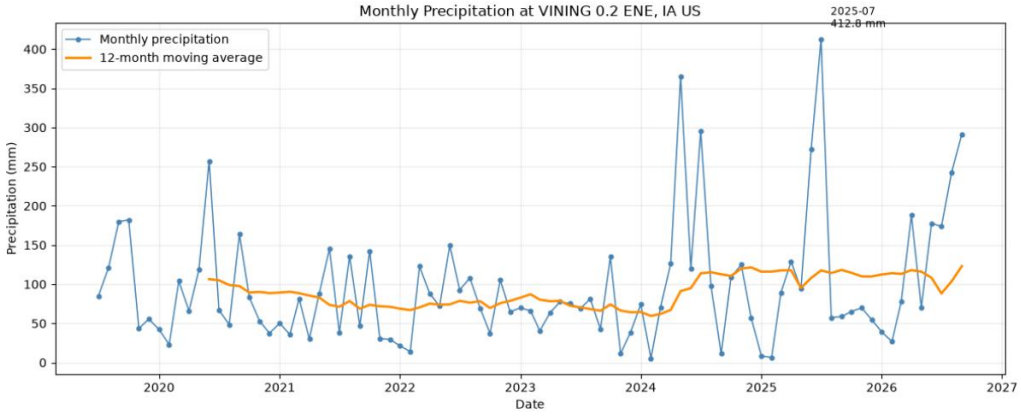
# 原始月降水量
ax.plot(
    monthly.index,
    monthly,
    color="steelblue",
    marker="o",
    markersize=3.5,
    linewidth=1.1,
    label="Monthly precipitation",
)

# 移动平均线
ax.plot(
    rolling_12.index,
    rolling_12,
    color="darkorange",
    linewidth=2,
    label="12-month moving average",
)

# 标出最大降水月份
max_row = df.loc[df["PRCP"].idxmax()]
ax.annotate(
    f"{max_row['DATE']:%Y-%m}\n{max_row['PRCP']:.1f} mm",
    xy=(max_row["DATE"], max_row["PRCP"]),
    xytext=(8, 10),
    textcoords="offset points",
    fontsize=8.5,
)

# 设置图形文字
ax.set_title("Monthly Precipitation at VINING 0.2 ENE, IA US")
ax.set_xlabel("Date")
ax.set_ylabel("Precipitation (mm)")
ax.legend()
ax.grid(alpha=0.25)
fig.tight_layout()

plt.show()
```



[ ]:
```
#4.3.1 描述统计与季节性
```

[4]:
```
# 检查 1：描述统计
summary = df["PRCP"].describe()
cv = df["PRCP"].std() / df["PRCP"].mean()
skewness = df["PRCP"].skew()

print("4.3 检查 1：描述统计")
display(summary.to_frame().T)
print(f"变异系数 CV = {cv:.3f}")
print(f"偏度 = {skewness:.3f}")

# 检查 2：完整性
print("\n4.3 检查 2：完整性")
print(f"有效月份 {len(df)} / 应有月份 {len(expected_months)}")
print(f"完整率 = {completeness:.2%}")

# 检查 3：月份季节性
df["MONTH"] = df["DATE"].dt.month

monthly_climatology = (
    df.groupby("MONTH")["PRCP"]
      .agg(["mean", "median", "std", "count"])
)

print("\n4.3 检查 3：月份季节性")
display(monthly_climatology.round(2))

wettest_month = monthly_climatology["mean"].idxmax()
driest_month = monthly_climatology["mean"].idxmin()

print(
    f"最湿月份：{wettest_month} 月，"
    f"平均 {monthly_climatology.loc[wettest_month, 'mean']:.2f} mm"
)
print(
    f"最干月份：{driest_month} 月，"
    f"平均 {monthly_climatology.loc[driest_month, 'mean']:.2f} mm"
)

# 绘制季节柱状图
month_labels = [
    calendar.month_abbr[m]
    for m in monthly_climatology.index
]

fig, ax = plt.subplots(figsize=(9, 4.5))
ax.bar(month_labels, monthly_climatology["mean"], color="steelblue")
ax.set_title("Average Monthly Precipitation")
ax.set_xlabel("Month")
ax.set_ylabel("Average precipitation (mm)")
ax.grid(axis="y", alpha=0.25)
fig.tight_layout()

plt.show()
```

```
4.3 检查 1：描述统计
```

	count	mean	std	min	25%	50%	75%	max
PRCP	87.0	96.11954	77.836544	5.6	45.05	72.1	121.9	412.8

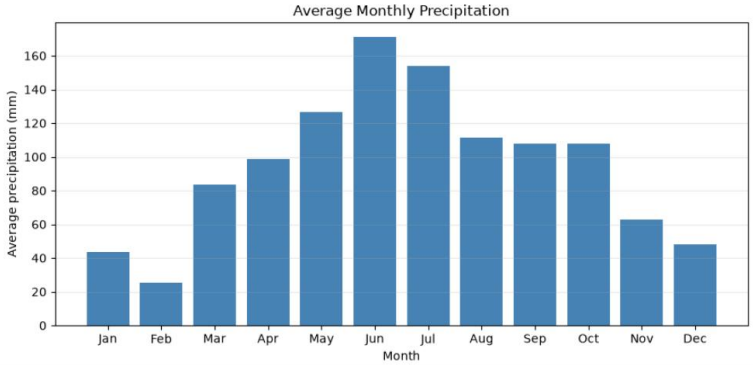
```
变异系数 CV = 0.810
偏度 = 1.885

4.3 检查 2：完整性
有效月份 87 / 应有月份 87
完整率 = 100.00%

4.3 检查 3：月份季节性
```

MONTH	mean	median	std	count
1	43.67	42.40	23.85	7
2	25.33	22.90	21.01	7
3	83.67	81.50	25.99	7
4	98.64	88.00	52.82	7
5	126.50	88.10	106.63	7
6	171.03	149.50	71.42	7
7	154.16	88.65	133.32	8
8	111.59	102.90	60.88	8
9	107.96	64.05	95.06	8
10	107.73	108.90	49.83	7
11	62.66	53.00	40.56	7
12	48.30	55.00	12.92	7

```
最湿月份：6 月，平均 171.03 mm
最干月份：2 月，平均 25.33 mm
```



[ ]:
```
#4.3.2 趋势、相关性和极端值
```

[6]:
```
# 检查 4：线性趋势
years = (df["DATE"] - df["DATE"].min()).dt.days / 365.25

slope, intercept = np.polyfit(years, df["PRCP"], 1)
time_corr = np.corrcoef(years, df["PRCP"])[0, 1]
r_squared = time_corr**2

print("4.3 检查 4：长期趋势")
print(f"每年变化：{slope:.2f} mm")
print(f"每十年变化：{slope * 10:.2f} mm")
print(f"R² = {r_squared:.3f}")
print(f"时间相关系数 = {time_corr:.3f}")

# 检查 5：PRCP 与降水日数的相关性
corr_table = (
    df[["PRCP", "DP01", "DP10", "DP1X", "EMXP"]]
    .corr()["PRCP"]
    .drop("PRCP")
    .to_frame("与 PRCP 的相关系数")
)

print("\n4.3 检查 5：相关性")
display(corr_table.round(3))
```

```
4.3 检查 4：长期趋势
每年变化：7.29 mm
每十年变化：72.87 mm
R² = 0.039
时间相关系数 = 0.197

4.3 检查 5：相关性
```

	与 PRCP 的相关系数
DP01	0.768
DP10	0.787
DP1X	0.870
EMXP	0.815

VINING 0.2 ENE 站 2019 年 7 月至 2025 年 12 月共有 78 个完整月份记录，月总降水量总体呈明显季节变化，6 月平均降水最多，2 月最少。月降水量与强降水日数相关性较高，说明降水总量受强降水事件影响明显。